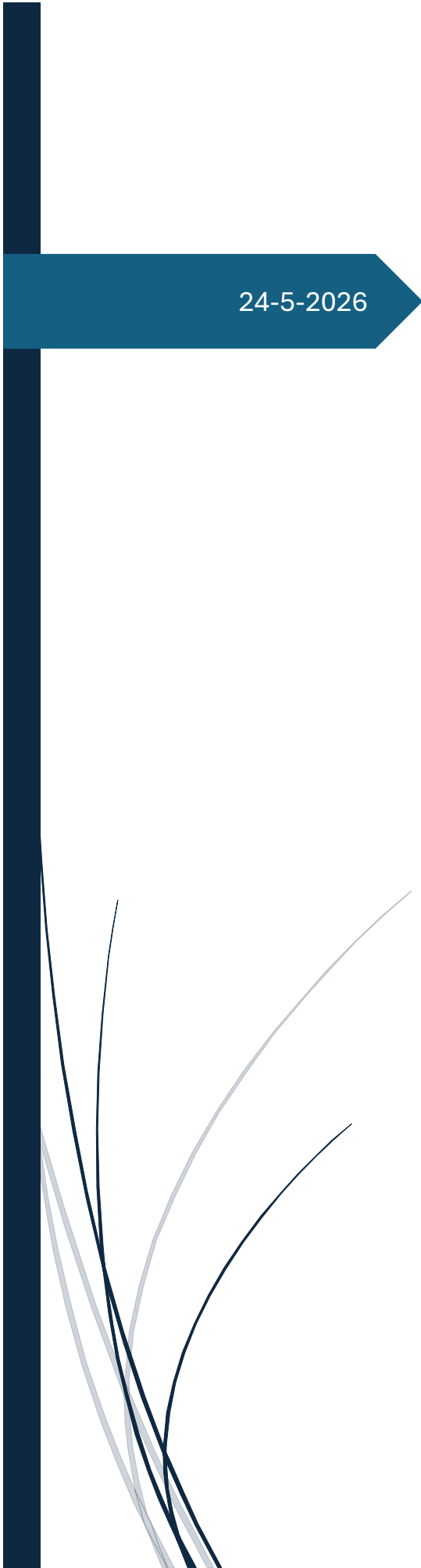


A thick dark blue vertical bar is positioned on the left side of the slide. A horizontal blue arrow points to the right from this bar, containing the date '24-5-2026'. Below the arrow, several thin, curved lines in dark blue and light grey sweep upwards from the bottom left towards the center of the slide.

24-5-2026

Predicción de resultados y Simulación Monte Carlo del Mundial FIFA 2026 utilizando Machine Learning

Práctico 1

Monica Castillo

UNIVERSIDAD DE EL SALVADOR, FACULTAD DE INGENIERÍA Y
ARQUITECTURA.

Desafío:

Desarrollar un pipeline de Machine Learning capaz de predecir la probabilidad de que cada una de las 48 selecciones nacionales se corone como campeona del Mundial de Fútbol FIFA 2026. Dado que un torneo eliminatorio es un proceso secuencial, el objetivo final no es solo darle un "1" al campeón, sino simular el torneo completo (Monte Carlo) basándose en las probabilidades de victoria de cada partido generadas por su modelo.

Objetivo:

Desarrollar un pipeline de Machine Learning capaz de predecir la probabilidad de que una selección nacional gane el Mundial FIFA 2026 mediante simulaciones Monte Carlo basadas en probabilidades de resultados de partidos.

Dataset elegidos:

Se utilizó la fuente de los resultados de partidos de mundiales/internacionales desde 1872-2024

- Resultados de Partidos Internacionales (1872-2024): GitHub - martj42/international_results.
 - Results.csv
 - El otro

Análisis de lo que tenemos:

En el archivo results.csv tenemos las variables

Nombre columna	Tipo de dato	Cantidad de datos	Breve descripción
Date	Object	49329	Fecha en la que se jugó el partido
Home_team	Object	49329	Nombre del equipo local
Away_team	Object	49329	Nombre del equipo visitante
Home_score	Float(64)	49257	Goles anotados por el equipo local
Away_score	Float(64)	49257	Goles anotados por el equipo visitante
Tournament	Object	49329	Nombre del torneo en el que se jugó el partido
City	Object	49329	Ciudad donde se jugó el partido
Country	Object	49329	País donde se jugó el partido

Neutral	Bool	49329	Indica si la cancha donde se jugó el partido, pertenece o nó al equipo local.
---------	------	-------	---

Limpieza de datos:

Observamos que existe una cantidad de nulos en home_score y away_score, de los cuales, debemos limpiarlo.

Una vez realizado la limpieza de nulos, procedemos a corregir los tipos de datos, en home_score y away_score, de tipo flotante a entero.

Creamos un campo para colocar las variables de quien ha ganado el partido, si el equipo visitante, el de casa o fue empate, a esta variable la llamaremos **target**, con los valores 0 si gana el equipo local, 1 si es empate, 2 si gana el equipo visitante.

Verificamos el balanceo del dataset:

Al ejecutar un counts para que nos muestre la cantidad de partidos ganados por el local, visitante o si fue empate, tenemos los datos

Target	Ganador	Count	%gane
0	Local	24,130	48.99%
1	Empate	13,925	28.27%
2	Visitante	11,202	22.74%
Total		49,257	

Con esto, podemos observar que el dataset presenta un ligero desbalance, con una mayor frecuencia de victorias si el equipo juega como local, lo cual sugiere la existencia de ventaja de localía incluso en partidos internacionales, no presenta desbalance extremo, ya que presenta una distribución moderada de clases.

En el data set tenemos 336 equipos, de los cuales, solo necesitamos 48, tenemos los países del mundial:

Canada	Mexico	United States	Argentina	Brazil
Colombia	Ecuador	Paraguay	Uruguay	Austria
Belgium	Bosnia and Herzegovina	Croatia	Czech Republic	England
France	Germany	Netherlands	Norway	Portugal
Scotland	Spain	Sweden	Switzerland	Turkey
Algeria	Cape Verde	Congo DR	Côte d'Ivoire	Egypt
Ghana	Morocco	Senegal	South Africa	Tunisia

Australia	Iran	Iraq	Japan	Jordan
South Korea	Qatar	Saudi Arabia	Uzbekistan	Curaçao
Haiti	Panama	New Zealand		

Observando que los países Congo DR y Côte d'Ivoire se encuentran en el dataset como DR Congo y Ivory Coast, con esto identificado, procedemos a normalizar los nombres de selecciones para garantizar consistencia entre fuentes de datos y evitar duplicados o entidades no deseadas.

- Luego, filtramos los equipos que se encuentran en el ranking y el vector construido.
- Verificamos que los 48 equipos estén dentro del filtro

```
#verificamos la lista
df_wc.shape
```

```
... (7501, 10)
```

```
df_wc["home_team"].nunique()
```

```
48
```

```
df_wc["away_team"].nunique()
```

```
... 48
```

Verificamos el balanceo nuevamente del dataset:

Al ejecutar un counts para los nuevos valores, nos muestra la cantidad de partidos ganados por el local, visitante o si fue empate, tenemos los datos

Target	Ganador	Count	%gane
0	Local	3,479	46.38%
1	Empate	2,124	28.31%
2	Visitante	1,898	25.30%
Total		7,501	

La distribución se mantiene consistente después de realizar el filtro de las selecciones de la FIFA que si irán al mundial, lo que nos indica que el dataset, preserva las características estadísticas.

Features

- **Construimos las victorias obtenidas por equipo:** unas de las variables que generamos es la proporción de partidos ganados por equipo, y el promedio de goles por partido.

Al generar un estadístico de los 10 equipos mas fuertes o con mayor obtenemos:

```
#vemos los 10 equipos mas fuertes
stats_df.sort_values("winrate", ascending=False).head
```

	winrate	avg_goals	matches
Brazil	0.580981	1.967311	673
Germany	0.515770	1.992579	539
Spain	0.507772	1.777202	386
Argentina	0.494236	1.766571	694
England	0.476190	1.918095	525
France	0.459839	1.630522	498
Mexico	0.436364	1.543434	495
Netherlands	0.412698	1.869048	504
Iran	0.408889	1.186667	225
Australia	0.408360	1.411576	311

Se incluyeron los features:

- **Matches:** como el numero de partidos jugados por selección, este nos servirá como indicador de confiabilidad de las métricas de rendimiento.
- **Winrate:** Nos dice que tan bueno o superior es el equipo, o la fuerza del equipo
- **Avg_goals:** Es el promedio de goles que ha hecho el equipo, por los partidos jugados.

Con ello, pasamos a realizar los últimos features

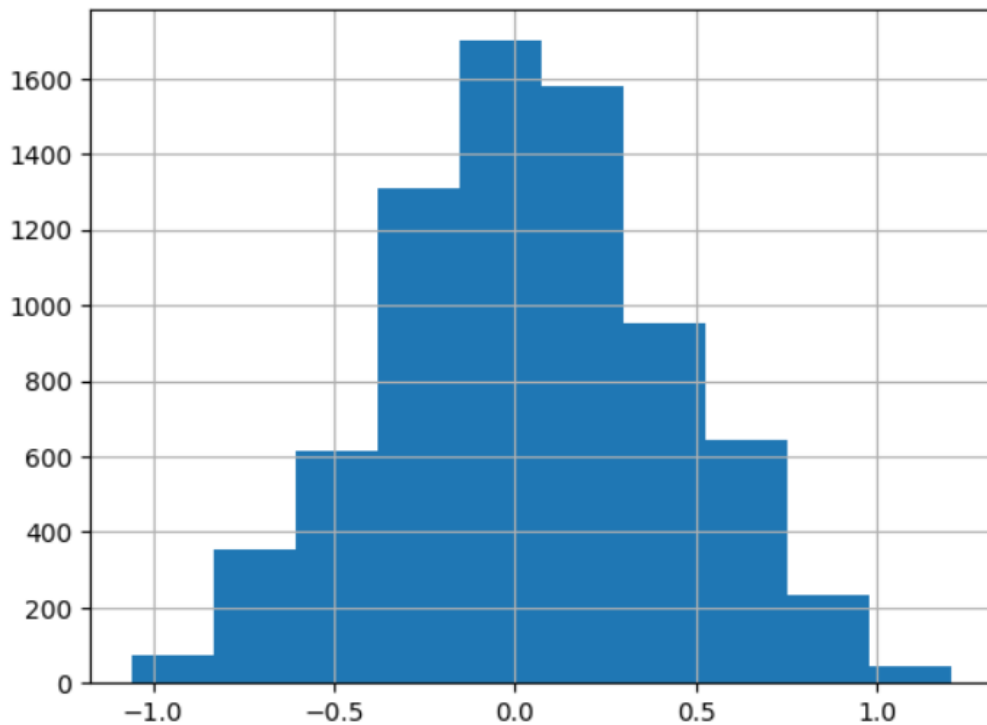
- Se utilizaron diferencias entre métricas de equipos para representar relaciones relativas en lugar de valores absolutos, permitiendo que el modelo aprenda

comparaciones directas entre selecciones, le decimos al modelo que tan superior es el equipo local con respecto al visitante.

- Se utilizó una transformación fila a fila para generar variables predictoras basadas en diferencias entre equipos, permitiendo representar cada partido como un vector numérico.
- Las variables generadas fueron analizadas mediante estadísticas descriptivas y visualización de su distribución, verificando su consistencia antes del entrenamiento del modelo.

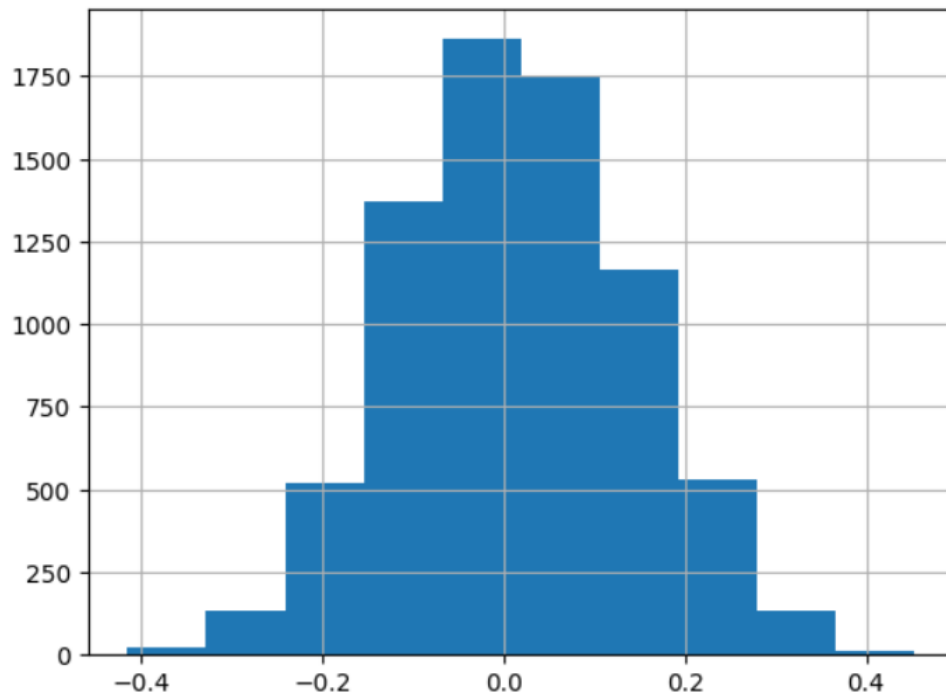
```
▶ #visualizamos la distribución rápida de featrures  
x["goals_diff"].hist()
```

*** <Axes: >



```
x["winrate_diff"].hist()
```

... <Axes: >



- Cuando mostramos los valores de X.head() tenemos:

```
X.head()
```

	winrate_diff	goals_diff	neutral	abs_winrate_diff	abs_goals_diff	home_stronger	strength_combined
0	-0.129734	-0.597885	0	0.129734	0.597885	0	0.077566
1	0.129734	0.597885	0	0.129734	0.597885	1	0.077566
2	-0.129734	-0.597885	0	0.129734	0.597885	0	0.077566
3	0.129734	0.597885	0	0.129734	0.597885	1	0.077566
4	-0.129734	-0.597885	0	0.129734	0.597885	0	0.077566

Cada fila representa un partido,

en la primera fila, podemos decir que, el visitante fue mas fuerte, anotó mas goles y el partido se jugó con localía.

En la segunda fila, podemos observar que, el local fue mas fuerte, anotó mas goles y se jugó con localía, y así sucesivamente por partido.

Con el X.describe() observamos el resumen estadístico:

 x.describe()

```
***
```

	winrate_diff	goals_diff	neutral	abs_winrate_diff	abs_goals_diff	home_stronger	strength_combined
count	7501.000000	7501.000000	7501.000000	7501.000000	7501.000000	7501.000000	7501.000000
mean	0.012098	0.036276	0.254366	0.104308	0.315013	0.536862	0.045745
std	0.130016	0.392282	0.435533	0.078543	0.236547	0.498673	0.066039
min	-0.415418	-1.061886	0.000000	0.000015	0.000332	0.000000	-0.024653
25%	-0.083839	-0.218947	0.000000	0.037651	0.123202	0.000000	0.003114
50%	0.010552	0.037162	0.000000	0.090752	0.266376	1.000000	0.018006
75%	0.103072	0.299349	1.000000	0.142483	0.461364	1.000000	0.061541
max	0.453464	1.208921	1.000000	0.453464	1.208921	1.000000	0.548202

- Cantidad: 7501 partidos
- Media: 0.012098, no hay tendencia a decir que “el local siempre gana”, no hay tendencia hacia un gran dominante o algún gigante dominante.
- Desviación estándar: 0.130016 con winrate_diff, nos dice que hay cambios moderados entre equipos.

Con estas podemos observar, que las variables construidas presentan una distribución centrada alrededor de cero, lo cual indica equilibrio entre las diferencias de rendimiento de los equipos y ausencia de sesgos estructurales en el dataset, no se observa que haya un equipo dominante.

Importar y entrenar los modelos

Importamos los modelos, Regresión Logística, Random Forest y XGBoost.

Separamos los datos 80% para el aprendizaje y el 20% para la evaluación.

- Modelo de Regresión Logística:

Al entrenar el modelo de regresión logística, configuramos un numero máximo de 1000 iteraciones, de las cuales, solo utilizó 32 para converger, indica estabilidad en la optimización y adecuada separación lineal aproximada en el espacio de características.

Al evaluar el modelo, obtenemos las métricas

variable	Precisión	Recall	F1- score	Support
0 – gana local	0.54	0.84	0.66	690
1 – Empate	0.00	0.00	0.00	363
2 – gana visitante	0.47	0.44	0.45	448
Acurracy			0.52	1501
Macro avg	0.34	0.43	0.37	1501
Weighted avg	0.39	0.52	0.44	1501

Log loss: 1.0163442641693878

Tenemos que el modelo acierta un 51%, el cual no es un resultado aceptable. Observamos que no está prediciendo en los empates, ya que la metrica de la precisión y el recall los tiene al 0, podemos observar que está aprendiendo lo fácil, que es el local gana vs el visitante.

El modelo presenta dificultades en la predicción de empates debido a la naturaleza estos son muy impredecibles, no siguen un patrón claro en los datos y dependen de factores aleatorios del partido, los resultados no siguen reglas claras en los datos que ingresamos.

- Modeo XGBoost

Al entrenar el modelo XGBoost, permitió capturar una pequeña mejora en la detección de empates, el rendimiento general sugiere que las variables utilizadas aun no son suficientes para modelar completamente la complejidad.

variable	Precisión	Recall	F1- score	Support
0 – gana local	0.54	0.80	0.64	690
1 – Empate	0.30	0.06	0.10	363

2 – gana visitante	0.48	0.43	0.40	448
Accuracy			0.51	1501
Macro avg	0.44	0.43	0.40	1501
Weighted avg	0.46	0.51	0.45	1501

Validación Cruzada

Al realizar la validación cruzada de ambos modelos, obtenemos los datos del accuracy promedio

- Regresión Logística: 0.5107
- XGBoost 0.5034

La Regresión Logística mostró un rendimiento ligeramente superior y más estable que XGBoost bajo validación cruzada. Esto sugiere que las variables utilizadas presentan relaciones predominantemente lineales y que el conjunto de características aún es limitado para explotar completamente el potencial de modelos más complejos.

Realizar la simulación Monte Carlo.

Resultados, ordenados de mayor a menor probabilidad de ganar el mundial

team	championship_probability	historical_winrate	avg_goals	matches
Brazil	14.95	58.10	1.97	673
Germany	11.64	51.58	1.99	539
England	8.27	47.62	1.92	525
Spain	8.02	50.78	1.78	386
Argentina	7.11	49.42	1.77	694

team	championship_probabil ity	historical_winr ate	avg_goa ls	matche s
Netherlan ds	4.88	41.27	1.87	504
France	4.56	45.98	1.63	498
Mexico	3.49	43.64	1.54	495
Sweden	3.33	38.54	1.71	467
Belgium	2.85	37.60	1.65	500
Austria	2.74	39.18	1.64	342
Australia	2.62	40.84	1.41	311
Croatia	2.07	38.60	1.39	171
Iran	1.90	40.89	1.19	225
Uruguay	1.78	37.66	1.43	608
Portugal	1.48	38.08	1.35	344
South Korea	1.43	38.08	1.23	386
Morocco	1.42	38.02	1.18	263
Czech Republic	1.35	36.36	1.31	132

team	championship_probabil ity	historical_winr ate	avg_goa ls	matche s
Japan	1.12	34.95	1.30	329
Scotland	1.05	34.65	1.32	381
Algeria	0.92	35.53	1.14	228
Turkey	0.89	34.58	1.18	240
Ghana	0.84	34.07	1.26	226
Côte d'Ivoire	0.82	34.06	1.27	229
United States	0.82	34.07	1.22	405
South Africa	0.77	32.94	1.22	170
Egypt	0.77	35.07	1.10	288
Colombia	0.72	36.26	1.15	364
Switzerlan d	0.67	29.12	1.36	443
Haiti	0.60	31.30	1.13	115
Tunisia	0.49	32.04	1.10	284
Iraq	0.47	32.30	1.04	257

team	championship_probabil ity	historical_winr ate	avg_goa ls	matche s
Senegal	0.43	33.95	0.99	215
Norway	0.39	24.78	1.26	343
Saudi Arabia	0.37	30.25	1.04	281
Paraguay	0.32	25.32	1.07	474
Canada	0.31	27.00	0.99	200
Uzbekista n	0.25	27.12	1.04	118
Qatar	0.25	26.48	0.98	219
Curaçao	0.18	20.00	1.10	60
Ecuador	0.16	22.60	0.98	323
New Zealand	0.10	16.56	0.94	151
Jordan	0.10	22.36	0.86	161
Bosnia and Herzegovi na	0.09	16.83	0.93	101
Congo DR	0.09	17.19	1.00	128

team	championship_probabil ity	historical_winr ate	avg_goa ls	matche s
Panama	0.09	12.75	0.76	149
Cape Verde	0.03	12.28	0.60	57

Repositorio:

<https://github.com/mcastilloh/desafio-practico1.git>

Código del proyecto

```
# -*- coding: utf-8 -*-
```

```
"""Practico 1.ipynb
```

Automatically generated by Colab.

Original file is located at

<https://colab.research.google.com/drive/16niT3FKOef0tjpZ-FhateWhP9v6046TW>

```
"""
```

```
from google.colab import files
```

```
carga = files.upload()
```

```
import pandas as pd
```

```
df = pd.read_csv("results.csv")
```

```
df.head()
```

```
df.info()
```

```
#limpiamos los datos de home score y away score ya que tienen nulos
```

```
df = df.dropna(subset=["home_score", "away_score"])
```

```
#verificamos que ya esté limpio
```

```
df.info()
```

#convertimos los tipos de datos home score y away score a enteros, ya que son los goles de local y visitante

```
df["home_score"] = df["home_score"].astype(int)
```

```
df["away_score"] = df["away_score"].astype(int)
```

#verificamos los cambios

```
df.info()
```

#vemos cuantos países se encuentran en la lista

#obtenemos todos los equipos que han jugado

```
teamsi = pd.unique(df[[
```

```
    'home_team',
```

```
    'away_team'
```

```
]].values.ravel())
```

```
len(teamsi)
```

#creamos el vector que obtendrá todos los equipos participantes

```
worldcup_teams = [
```

```
    "Canada", "Mexico", "United States", "Argentina", "Brazil",
```

```
    "Colombia", "Ecuador", "Paraguay", "Uruguay", "Austria",
```

```
    "Belgium", "Bosnia and Herzegovina", "Croatia", "Czech Republic", "England",
```

```
    "France", "Germany", "Netherlands", "Norway", "Portugal",
```

```
    "Scotland", "Spain", "Sweden", "Switzerland", "Turkey",
```

```
    "Algeria", "Cape Verde", "Congo DR", "Côte d'Ivoire", "Egypt",
```

```
    "Ghana", "Morocco", "Senegal", "South Africa", "Tunisia",
```

```
    "Australia", "Iran", "Iraq", "Japan", "Jordan",
```



```
"South Korea","Qatar", "Saudi Arabia", "Uzbekistan", "Curaçao",  
"Haiti", "Panama", "New Zealand"  
]
```

#normalizamos los nombres de los equipos

```
df["home_team"] = df["home_team"].replace({  
    "DR Congo": "Congo DR",  
    "Ivory Coast": "Côte d'Ivoire",  
})
```

```
df["away_team"] = df["away_team"].replace({  
    "DR Congo": "Congo DR",  
    "Ivory Coast": "Côte d'Ivoire",  
})
```

#definimos un campo donde nos guarde el ganador local, visitante o si fue empate

```
def get_result(row):  
    if row["home_score"] > row["away_score"]:  
        return 0 # local gana  
    elif row["home_score"] < row["away_score"]:  
        return 2 # visitante gana  
    else:  
        return 1 # empate
```

```
df["target"] = df.apply(get_result, axis=1)
```

#filtramos los equipos pertenecientes al ranking de la FIFA

```
df_wc = df[  
    df["home_team"].isin(worldcup_teams) &  
    df["away_team"].isin(worldcup_teams)  
].copy()
```

```
sorted(df_wc["home_team"].unique())
```

```
sorted(df_wc["away_team"].unique())
```

#verificamos la lista

```
df_wc.shape
```

```
df_wc["home_team"].nunique()
```

```
df_wc["away_team"].nunique()
```

#verificamos la creación de la columna

```
df_wc[["home_team",  
    "home_score",  
    "away_team",  
    "away_score",  
    "target"]].head()
```

#verificamos la cantidad de victorias por visitante, local y empates

```
df_wc["target"].value_counts()
```

#verificamos la proporción de las victorias obtenidas en porcentaje

```
df_wc["target"].value_counts(normalize=True) * 100
```

#calculamos las estadísticas por equipo

```
teams = pd.unique(df_wc[[
```

```
    'home_team',
```

```
    'away_team'
```

```
]])
```

```
len(teams)
```

#inicializamos un contenedor vacío.

```
team_stats = {}
```

#calculamos las estadísticas por equipo

```
for team in teams:
```

#calculamos cuando juega en local y cuando en visitante

```
home = df_wc[df_wc["home_team"] == team]
```

```
away = df_wc[df_wc["away_team"] == team]
```

#calculamos las victorias que tuvo el equipo

```
wins = (
```

```
    (home["home_score"] > home["away_score"]).sum() +
```

```
    (away["away_score"] > away["home_score"]).sum()
```

```
)
```

#caluclamos cuantos partidos jugó el equipo.

```
matches = len(home)+len(away)
```

#obtenemos la cantidad de goles

```
goals_scored = home["home_score"].sum() + away["away_score"].sum()
```

#obtenemos la proporción de partidos ganados por equipo

```
winrate = wins / matches if matches > 0 else 0
```

#caluclamos los goles promedio por partido

```
avg_goals = goals_scored / matches if matches > 0 else 0
```

#guardamos en el diccionario

```
team_stats[team] = {  
    "winrate" : winrate,  
    "avg_goals": avg_goals,  
    "matches" : matches  
}
```

```
set(team_stats.keys()) - set(worldcup_teams)
```

#verificamos los equipos mas fuertes

```
stats_df = pd.DataFrame.from_dict(team_stats, orient="index")
```

```
stats_df.sort_values("winrate", ascending=False).head(10)
```

#verificamos los datos

```
stats_df = pd.DataFrame.from_dict(team_stats, orient="index")
stats_df.head()
```

#vemos los 10 equipos mas fuertes

```
stats_df.sort_values("winrate", ascending=False).head(10)
```

```
def get_features(row):
```

#extraemos los equipos

```
home = row["home_team"]
```

```
away = row["away_team"]
```

#calculamos, winrate: diferencia de fuerza histórica

#avg_goals: diferencia de ataque

#neutral: campo del local, o no es de ninguno

```
return pd.Series([
    team_stats[home]["winrate"] - team_stats[away]["winrate"],
    team_stats[home]["avg_goals"] - team_stats[away]["avg_goals"],
    row["neutral"]
])
```

#aplicamos los features al dataset, convertimos los partidos en numeros

#predecimos quien es mejor+que tan parejo es el partido

```
X = df_wc.apply(get_features, axis=1)
```

```
X.columns = ["winrate_diff", "goals_diff", "neutral"]
```

```
X["abs_winrate_diff"] = X["winrate_diff"].abs()
```

```
X["abs_goals_diff"] = X["goals_diff"].abs()
```

```
X["home_stronger"] = (X["winrate_diff"]>0).astype(int)
```

```
X["strength_combined"] = X["winrate_diff"] * X["goals_diff"]
```

```
X["match_balance"] = (X["abs_winrate_diff"] + X["abs_goals_diff"])
```

```
#resultado
```

```
y = df_wc["target"]
```

```
#convertimos el X a entero
```

```
X=X.fillna(0)
```

```
X["neutral"] = X["neutral"].astype(int)
```

```
X.head()
```

```
X.describe()
```

```
#debemos visualizar el resultado conjunto
```

```
pd.concat([X, df_wc["target"]], axis=1).head()
```

```
#visualizamos la distribución rápida de featrures
```

```
X["goals_diff"].hist()
```

```
X["winrate_diff"].hist()
```

```
#importamos los modelos Logistic Regression, Random Forest y XGBoost
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, log_loss
```

```
#separamos los datos 80% para aprendizaje, 20% para evaluar
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
#entrenamos el modelo de regresión logística
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
y_pred_lr2 = model.predict(X_test)
```

```
y_prob_lr2 = model.predict_proba(X_test)
```

```
print("Logistic regression - features")
```

```
print(classification_report(y_test, y_pred_lr2))
```

```
print(log_loss(y_test, y_prob_lr2))
```

```
model.coef_
```

```
#realizamos las predicciones
```

```
y_pred = model.predict(X_test)
```

```
y_prob = model.predict_proba(X_test)
```

```
#evaluamos
```

```
print(classification_report(y_test, y_pred))
```

```
model.n_iter_
```

```
#importamos el modelo XGBoost
```

```
from xgboost import XGBClassifier
```

```
#entrenamos el modelo
```

```
xgb_model = XGBClassifier(
```

```
    n_estimators=200,
```

```
    max_depth=4,
```

```
    learning_rate=0.05,
```

```
    subsample=0.8,
```

```
    random_state=42
```

```
)
```

```
xgb_model.fit(X_train, y_train)
```

```
#realizamos las predicciones
```

```
y_pred_xgb = xgb_model.predict(X_test)
```

```
y_prob_xgb = xgb_model.predict_proba(X_test)
```

```
#Evaluamos el modelo
```

```
print(classification_report(y_test, y_pred_xgb))
```

```
print(log_loss(y_test, y_prob_xgb))
```


#realizar la validación cruzada

```
from sklearn.model_selection import cross_val_score
```

#validación a la regresión logística

```
cv_scores_lr = cross_val_score(
```

```
    model,
```

```
    X,
```

```
    y,
```

```
    cv=5,
```

```
    scoring="accuracy"
```

```
)
```

```
print("Logistic Regression - Validación Cruzada:")
```

```
print(cv_scores_lr)
```

```
print("Average Accuracy:")
```

```
print(cv_scores_lr.mean())
```

#Validacion al XGBoost

```
cv_scores_xgb = cross_val_score(
```

```
    xgb_model,
```

```
    X,
```

```
    y,
```

```
    cv=5,
```

```
    scoring="accuracy"
```

```
)
```

```
print("XGBoost - Validación cruzada:")
```

```
print(cv_scores_xgb)
```

```
print("Average Accuracy:")
```

```
print(cv_scores_xgb.mean())
```

```
#simulación Monte Carlo
```

```
teams = list(worldcup_teams)
```

```
#Creamos un prtido
```

```
import random
```

```
import numpy as np
```

```
def simulate_match(team1, team2):
```

```
    winrate_diff=(  
        team_stats[team1]["winrate"] -  
        team_stats[team2]["winrate"]  
    )
```

```
    goals_diff = (  
        team_stats[team1]["avg_goals"] -  
        team_stats[team2]["avg_goals"]  
    )
```

```
features = pd.DataFrame([{
```

```
"winrate_diff": winrate_diff,
"goals_diff": goals_diff,
"neutral": 1,
"abs_winrate_diff": abs(winrate_diff),
"abs_goals_diff": abs(goals_diff),
"home_stronger": int(winrate_diff > 0),
"strength_combined": winrate_diff * goals_diff,
"match_balance": (
    abs(winrate_diff) + abs(goals_diff)
)
})
```

#probabilidades del modelo

```
probs = model.predict_proba(features)[0]
```

#Elegimos resultado aleatorio

```
outcome = np.random.choice(
    [0, 1, 2],
    p=probs
)
```

#devolvemos al ganador

```
if outcome == 0:
    return team1
elif outcome == 2:
    return team2
```

else:

return random.choice([team1, team2])

#probamos un partido

simulate_match("Argentina", "Brazil")

#simulamos todo el torneo para sacar las probabilidades del campeon

#simulamos la primera ronda, tenemos 48 equipos, de los cuales saldrán 24 ganadores

#segunda ronda, entran 24 equipos, salen 12 ganadores

#tercera ronda, 12 equipos, 8 ganadores

#cuartos de final 8 equipos, 4 ganadores

#semi final 4 equipos, 2 ganadores

#Final 2 equipos, 1 ganador

def **simulate_tournament**(teams):

teams = teams.copy()

random.shuffle(teams)

while **len**(teams) > 1:

if **len**(teams) % 2 == 1:

teams.append(random.choice(teams))

```
winners = []
```

```
for i in range(0, len(teams), 2):
```

```
    team1 = teams[i]
```

```
    team2 = teams[i + 1]
```

```
    winner = simulate_match(team1, team2)
```

```
    winners.append(winner)
```

```
teams = winners
```

```
return teams[0]
```

```
#probamos un torneo
```

```
simulate_tournament(teams)
```

```
#Simulamos 10000 mundiales
```

```
champions = []
```

```
for i in range(10000):
```

```
    champion = simulate_tournament(teams)
```

```
    champions.append(champion)
```

#contamos los campeones

from collections import Counter

results = Counter(champions)

#vemos el top 10 de campeones

results.most_common(10)

#convertimos en probabilidades

total_simulations = **len**(champions)

final_results = []

for team in worldcup_teams:

 championships = results.get(team, 0)

 probability = (

 championships / total_simulations

) * 100

 final_results.append({

 "team": team,

```
"championship_probability":
```

```
    round(probability, 2),
```

```
"historical_winrate":
```

```
    round(team_stats[team]["winrate"] * 100, 2),
```

```
"avg_goals":
```

```
    round(team_stats[team]["avg_goals"], 2),
```

```
"matches":
```

```
    team_stats[team]["matches"]
```

```
})
```

```
final_df = pd.DataFrame(final_results)
```

```
final_df.sort_values(
```

```
    "championship_probability",
```

```
    ascending=False
```

```
)
```